

PROGRAMACIÓN III

Resumen completo para el primer parcial

Apunte unificado y ampliado de Git, ramas, web, HTML, CSS, modelo de caja, JavaScript y métodos de arreglos. Está pensado para estudiar todo seguido, entender cada concepto y repasar sin depender de los apuntes originales.

Orden del resumen: Introducción a Git, Ramas en Git, Introducción a la Web, Introducción a HTML, Introducción a CSS, Elementos HTML de bloque y en línea, El modelo de caja, Introducción a JavaScript y Métodos de arreglos.

Objetivo: no solo listar definiciones, sino explicar qué significa cada concepto, para qué sirve, cómo se relaciona con los demás y qué errores comunes conviene evitar en el parcial.

Cómo estudiar este apunte

La mejor forma de usarlo es en tres vueltas. Primera vuelta: leer cada tema para entender la idea general. Segunda vuelta: detenerse en ejemplos, tablas y comparaciones. Tercera vuelta: intentar responder con palabras propias las preguntas del repaso final.

1. Git: problema que resuelve, tipos de VCS, estados y comandos.
2. Ramas: punteros, HEAD, checkout, merge y conflictos.
3. Web: Internet, Web, URL, IP, DNS, HTTP, cliente y servidor.
4. HTML: estructura, elementos, atributos, anidación y semántica básica.
5. CSS: reglas, selectores, cascada, herencia, especificidad y propiedades frecuentes.
6. Bloque y línea: comportamiento por defecto y relación con `display`.
7. Modelo de caja: contenido, padding, border, margin y `box-sizing`.
8. JavaScript: variables, tipos, condicionales, bucles, funciones, objetos y módulos.
9. Métodos de arreglos: callbacks, transformaciones, búsquedas y acumulación.

1. Introducción a Git

Git es un sistema de control de versiones distribuido. Para entender por qué existe, primero hay que pensar en un problema simple: cuando varias personas trabajan sobre archivos, o incluso una sola persona trabaja durante varios días, es muy fácil perder cambios, sobrescribir trabajo útil o no recordar qué versión era la correcta. Un sistema de control de versiones aparece justamente para registrar el historial del proyecto y permitir volver atrás cuando haga falta.

¿Qué es un sistema de control de versiones?

Es una herramienta que registra los cambios realizados en uno o varios archivos a lo largo del tiempo. Eso permite recuperar versiones anteriores, comparar estados, saber quién hizo un cambio y mantener una historia ordenada del proyecto. En otras palabras, no se limita a "guardar archivos", sino que guarda la evolución del trabajo.

¿Para qué sirve?

- Volver a una versión anterior de un archivo o del proyecto completo.
- Comparar cambios entre distintos momentos.
- Detectar cuándo se introdujo un error.
- Saber quién modificó determinada parte del código.
- Ordenar el trabajo en equipo.

Problemas que evita

- Tener múltiples carpetas tipo "final", "final2", "ahora sí final".
- Perder código al sobrescribir archivos.
- No saber qué cambio rompió algo.
- No poder coordinar trabajo entre varias personas.

Tipos de VCS

Los apuntes distinguen tres grandes tipos de sistemas de control de versiones. La clasificación es importante porque muestra cómo fue evolucionando la forma de trabajar con proyectos.

Tipo	Cómo funciona	Ventaja principal	Limitación principal
Local	El historial se guarda en la misma computadora.	Es simple.	No favorece el trabajo colaborativo y es riesgoso si falla el equipo.
Centralizado	Hay un servidor central con el historial y cada usuario trabaja a partir de él.	Ordena mejor el trabajo en equipo.	Depende mucho del servidor central.
Distribuido	Cada usuario tiene una copia completa del repositorio y del historial.	Permite trabajar localmente con gran autonomía.	Exige entender mejor el flujo de sincronización.

¿Qué vuelve especial a Git?

Git pertenece al grupo distribuido. Eso significa que cada persona tiene una copia completa del repositorio, no solo una carpeta con archivos actuales. Gracias a eso, muchas operaciones se hacen localmente, con rapidez y sin depender de conexión constante a un servidor.

- **Alto rendimiento:** muchas operaciones son locales.
- **Diseño simple:** la idea de repositorio, commit y rama es potente pero compacta.
- **Soporte para desarrollo no lineal:** se puede trabajar en ramas sin mezclar todo inmediatamente.
- **Integridad:** los objetos se identifican con hashes, lo que ayuda a detectar modificaciones y preservar consistencia.

Cómo piensa Git los cambios

Un error común es imaginar que Git guarda una larga lista de diferencias sueltas. El material aclara que Git trabaja con **instantáneas**. Cada commit representa el estado del proyecto en un momento concreto. Eso simplifica muchas operaciones mentales: cuando hacés un commit, Git registra "así estaba el proyecto acá", no solo "cambió esta línea".

Integridad y hash

Git identifica objetos mediante una suma de verificación, tradicionalmente expresada como una cadena hexadecimal larga. Esto significa que cada commit, archivo u objeto importante queda identificado por un valor calculado en función de su contenido. La idea clave para el parcial no es memorizar el algoritmo, sino entender que Git usa ese identificador para garantizar consistencia y rastreabilidad.

Estados de los archivos

Una parte central del tema es el flujo de trabajo. En Git, los archivos atraviesan distintos estados mientras trabajás sobre ellos.

- **Modified:** el archivo cambió, pero esos cambios todavía no fueron preparados para un commit.
- **Staged:** el archivo ya fue marcado para incluirse en el próximo commit.
- **Committed:** el cambio quedó guardado en el repositorio local.

Idea importante: editar un archivo no significa automáticamente que Git ya lo va a guardar en el próximo commit. Primero hay que pasarlo por el área de preparación con `git add .`

Espacios de trabajo de Git

Aunque el material lo presenta con imágenes, conceptualmente conviene pensarlo así:

1. **Working directory:** donde están tus archivos actuales y donde vos editás.
2. **Staging area:** donde elegís qué cambios formarán parte del próximo commit.
3. **Repositorio:** donde quedan guardados los commits ya confirmados.

Este punto suele aparecer mucho en parciales porque explica por qué a veces un archivo está modificado pero todavía no "entra" en el commit.

Comandos básicos

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@mail.com"
git config --global core.editor "code --wait"
git config --list

git init
git status
git status --short
git add archivo.txt
git add .
git commit -m "Mensaje del commit"
git log
git log -p
git log -- archivo.txt
git remote -v
```

```
git remote add origin URL
git fetch
git pull
git push
```

Qué hace cada comando clave

- `git init` crea un repositorio Git en la carpeta actual.
- `git status` muestra el estado del proyecto: archivos nuevos, modificados o preparados.
- `git add` manda cambios al área de preparación.
- `git commit` confirma el estado preparado y crea un punto de historial.
- `git log` permite recorrer la historia del proyecto.
- `git fetch` descarga cambios remotos sin fusionarlos.
- `git pull` descarga y además integra cambios.
- `git push` envía commits locales al repositorio remoto.

Otros comandos frecuentes

El material también menciona operaciones como `git commit --amend`, útil para corregir el último commit. La idea no es abusar de ese comando, sino entender que Git permite ajustar la historia reciente mientras todavía no se haya compartido o mientras el flujo de trabajo lo permita.

Error conceptual común: pensar que Git “guarda solo cuando guardo el archivo”. No. Guardar el archivo en el editor y hacer un commit son cosas distintas. Guardar actualiza el archivo en el disco; el commit actualiza el historial de Git.

2. Ramas en Git

Una vez entendidos los commits, aparece uno de los conceptos más potentes de Git: las ramas. Las ramas permiten trabajar en distintas líneas de desarrollo sin mezclar todo inmediatamente. Son fundamentales cuando querés desarrollar una funcionalidad nueva, probar una idea o corregir un error sin tocar la versión principal del proyecto.

¿Qué es una rama?

Una rama es un **puntero** que señala un commit. Esta definición es muy importante. No es una copia completa del proyecto, no es una carpeta aparte y no es un archivo raro. Es simplemente

una referencia que marca “esta línea de trabajo llega hasta este commit”. Cuando hacés un nuevo commit en esa rama, el puntero avanza.

Por qué Git maneja ramas tan rápido

En otros sistemas, crear una rama puede requerir copiar gran parte del proyecto. En Git no pasa eso porque la rama es solo una referencia liviana a un commit. Como Git ya tiene las instantáneas y la relación entre commits, crear una rama nueva es barato y rápido. Por eso ramificar no es una excepción: es parte normal del trabajo.

HEAD

`HEAD` es una referencia especial que indica dónde estás parado actualmente. Si estás en `main` o `master`, `HEAD` apunta a esa rama. Si te cambiás a `testing`, `HEAD` pasa a apuntar a `testing`. En términos prácticos, `HEAD` marca cuál es la rama activa.

Operaciones básicas

```
git branch testing
git checkout testing
git checkout master
git checkout -b iss53
git log --oneline --decorate
git log --oneline --decorate --graph --all
```

- `git branch testing` crea la rama `testing`.
- `git checkout testing` cambia a esa rama.
- `git checkout -b iss53` crea y cambia en el mismo paso.
- `git log --graph --all` ayuda a visualizar la divergencia entre ramas.

Qué pasa al cambiar de rama

Cambiar de rama no significa solo mover un puntero interno. Git también actualiza el directorio de trabajo para que los archivos visibles coincidan con la instantánea correspondiente a esa rama. Por eso, después de hacer checkout, podés ver archivos distintos o contenidos distintos según la rama a la que te cambiaste.

Historial divergente

Si desde un commit común creás dos ramas y hacés commits diferentes en cada una, el historial diverge. Eso no es un problema: es justamente lo que permite trabajar en paralelo. El punto importante es que cada rama conserva su línea de evolución hasta que decidís fusionarlas.

Merge

Fusionar significa integrar cambios de una rama en otra. La idea general es: te posicionás en la rama destino y ejecutás `git merge` con la rama origen.

```
git checkout master
git merge iss53
git branch -d iss53
```

Si no hubo divergencia compleja, Git puede hacer una integración simple. Si ambas ramas avanzaron por separado, Git puede necesitar un merge de tres vías y crear un commit especial de fusión.

Escenario clásico: funcionalidad + hotfix

El apunte muestra un flujo muy común:

1. Se crea una rama para desarrollar una funcionalidad, por ejemplo `iss53`.
2. Se trabaja ahí y se hacen commits.
3. Aparece un error urgente en producción.
4. Se vuelve a la rama principal y se crea una rama `hotfix`.
5. Se corrige el problema, se fusiona el hotfix y luego se retoma el trabajo anterior.

Este ejemplo muestra por qué las ramas son tan útiles: permiten interrumpir una línea de trabajo sin mezclar todo ni perder contexto.

Conflictos de fusión

Si dos ramas modificaron la misma parte de un archivo, Git no puede decidir automáticamente qué versión dejar. Ahí aparece un conflicto de merge. En ese caso, Git marca el archivo, vos revisás manualmente qué contenido corresponde, guardás la resolución y luego confirmás el merge.

Idea de parcial: un conflicto no significa que Git "falló". Significa que Git detectó una ambigüedad que necesita decisión humana.

Precaución al hacer checkout

Si tenés cambios sin confirmar y esos cambios chocan con la rama a la que querés pasar, Git puede impedir el cambio para evitar que pierdas trabajo. Por eso suele recomendarse tener el directorio limpio antes de cambiar de rama.

3. Introducción a la web

Antes de programar páginas, conviene entender el entorno donde esas páginas existen. La Web es un sistema que funciona sobre Internet y permite que los usuarios accedan a información mediante navegadores. Este tema sirve como base para ubicar a HTML, CSS y JavaScript en el ecosistema general.

Internet y Web no son lo mismo

Internet es la red global que conecta dispositivos. La Web, en cambio, es uno de los servicios que funciona sobre esa red. Dicho simple: Internet es la infraestructura; la Web es una forma de usar esa infraestructura para acceder a páginas y aplicaciones.

¿Qué es la World Wide Web?

Es un sistema para publicar, acceder y relacionar información mediante recursos identificados por URLs. Cuando alguien abre un sitio en el navegador, lo que está haciendo es pedir un recurso web a través de Internet.

Elementos clave del funcionamiento web

- **URL:** dirección del recurso que se quiere solicitar.
- **IP:** dirección numérica de una máquina en la red.
- **Dominio:** nombre más fácil de recordar que representa una dirección.
- **DNS:** sistema que traduce dominio a IP.
- **HTTP:** protocolo de comunicación entre cliente y servidor en la Web.

Cómo funciona una solicitud

1. El usuario escribe una URL o hace clic en un enlace.
2. El navegador interpreta esa dirección.
3. Se consulta el DNS para obtener la IP asociada al dominio.
4. El navegador se comunica con el servidor usando HTTP.
5. El servidor responde con HTML, CSS, JavaScript, imágenes u otros recursos.
6. El navegador interpreta todo eso y construye la página visible.

Modelo cliente-servidor

La Web funciona bajo un modelo de pedido y respuesta. El cliente pide un recurso y el servidor responde. El cliente no tiene que saber todos los detalles de cómo se generó esa respuesta; solo necesita recibirla e interpretarla.

Lado del cliente	Lado del servidor
Se ejecuta principalmente en el navegador.	Se ejecuta en una computadora remota preparada para responder solicitudes.
Se ocupa de mostrar interfaz e interacción.	Se ocupa de procesar datos, aplicar lógica y servir recursos.
Usa HTML, CSS y JavaScript.	Puede usar bases de datos, autenticación, APIs y lógica de negocio.
También se lo llama front-end.	También se lo llama back-end.

Navegador

El navegador es el programa que pide recursos web, interpreta el contenido y lo muestra. No solo dibuja páginas: también administra historial, seguridad, descargas y herramientas de desarrollo. Este punto es importante porque HTML, CSS y JavaScript no “se ven” solos; necesitan un navegador que los interprete.

Relación con los temas siguientes: HTML define la estructura del contenido que recibe el navegador, CSS define su presentación y JavaScript agrega comportamiento en el lado del cliente.

4. Introducción a HTML

HTML significa **HyperText Markup Language**. Es el lenguaje de marcado que se usa para definir la estructura y el significado del contenido de una página. No se usa para programar lógica ni para diseñar visualmente la interfaz: su función principal es organizar la información para que el navegador sepa qué es cada cosa.

Por qué se lo llama lenguaje de marcado

Se lo llama así porque usa **marcas o etiquetas** para delimitar partes del contenido y darles significado. El texto solo no le dice demasiado al navegador; en cambio, si se lo marca como párrafo, encabezado, enlace o lista, el navegador ya puede interpretarlo correctamente.

Relación HTML, CSS y JavaScript

HTML

CSS

JavaScript

HTML define qué es cada parte del contenido. **CSS** define cómo se ve. **JavaScript** define cómo se comporta. Esta división de responsabilidades es la base del desarrollo web.

Primer ejemplo y estructura básica

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Mi primera página</title>
  </head>
  <body>
    <h1>Hola mundo</h1>
    <p>Este es mi primer documento HTML.</p>
  </body>
</html>
```

Este ejemplo ya muestra una idea importante: el documento no es una lista de etiquetas sueltas, sino una estructura jerárquica donde unas partes contienen a otras.

Qué significa cada parte

- `<!DOCTYPE html>` : indica que el documento está escrito según el estándar actual de HTML.
- `<html>` : envuelve todo el documento.
- `lang="es"` : indica el idioma principal del contenido, lo cual ayuda en accesibilidad y herramientas del navegador.
- `<head>` : contiene metadatos, como el título, la codificación de caracteres, enlaces a CSS y referencias a scripts.
- `<body>` : contiene el contenido visible de la página.

Etiqueta, elemento y atributo

Estos tres conceptos están relacionados, pero no son exactamente lo mismo:

- **Etiqueta:** la marca entre corchetes angulares, por ejemplo `<p>`.
- **Elemento:** la unidad completa formada por apertura, contenido y cierre cuando corresponde.

- **Atributo:** información adicional escrita en la etiqueta de apertura, por ejemplo `href` , `src` o `alt` .

Ejemplo: en `<a href="https://ejemplo.com">Ir</a>` , el elemento es el enlace completo, `href` es un atributo y la URL es el valor de ese atributo.

Anatomía de un elemento HTML

En muchos elementos pueden distinguirse cuatro partes:

- Etiqueta de apertura.
- Contenido.
- Etiqueta de cierre.
- Atributos, cuando hacen falta.

Por ejemplo, en `<p class="destacado">Texto</p>` , `p` es el nombre del elemento, `class` es un atributo, `"destacado"` es su valor y `Texto` es el contenido.

Elementos contenedores y elementos vacíos

Algunos elementos contienen texto u otros elementos, y por eso se llaman **contenedores**. Otros no envuelven contenido y se usan de forma independiente; suelen llamarse **elementos vacíos**.

- Contenedores: `<p>` , `<div>` , `<section>` .
- Vacíos: `<br>` , `<img>` , `<meta>` , `<input>` .

Anidación

Anidar significa colocar un elemento dentro de otro. HTML es jerárquico, así que la anidación correcta es fundamental. Si abris un elemento dentro de otro, también tenés que cerrarlo respetando ese orden. Una anidación incorrecta rompe la estructura lógica del documento y puede provocar interpretaciones inesperadas.

Espacios, saltos de línea e indentación

En HTML, varios espacios seguidos y muchos saltos de línea suelen **colapsarse** al mostrarse en pantalla. Esto quiere decir que el navegador no respeta el formato visual del código fuente como si fuera texto literal. Por eso, la indentación y los saltos se usan principalmente para que el código sea legible para quien lo escribe.

Elementos de uso general

El material dedica una sección importante a las etiquetas más frecuentes. Conviene conocer no solo el nombre de la etiqueta, sino también qué representa y cuándo conviene usarla.

Encabezados

Los encabezados se definen con las etiquetas `<h1>` a `<h6>`. Representan niveles de jerarquía dentro del contenido.

- `<h1>` es el encabezado más importante.
- `<h6>` es el de menor jerarquía.

```
<h1>Título principal</h1>
<h2>Subtítulo</h2>
<h3>Sección interna</h3>
```

No conviene usarlos solo porque “se ven más grandes” o “más chicos”. Deben reflejar la estructura lógica del texto.

Párrafos

Los párrafos se definen con la etiqueta `<p>`. Cada párrafo representa una unidad de texto separada.

```
<p>Esto es un párrafo.</p>
<p>Esto es otro párrafo.</p>
```

Saltos de línea

La etiqueta `<br>` inserta un salto de línea sin crear un nuevo párrafo.

```
<p>
  Línea 1<br>
  Línea 2<br>
  Línea 3
</p>
```

No debe usarse para separar bloques enteros ni para “acomodar” el diseño. Para eso se usan otras etiquetas y CSS.

Texto preformateado

La etiqueta `<pre>` muestra texto respetando espacios y saltos de línea. Es útil cuando el espaciado tiene significado, por ejemplo en código o salidas de consola.

```
<pre>
Línea 1
```

```
Línea 2 con indentación
  Línea 3 con más indentación
</pre>
```

Formato básico de texto

HTML incluye etiquetas para marcar énfasis, importancia o significado especial en fragmentos de texto.

```
<p>
  Texto <strong>importante</strong>,
  texto <em>enfaticado</em>
  y texto <mark>resaltado</mark>.
</p>
```

- `<strong>` : importancia fuerte.
- `<em>` : énfasis.
- `<mark>` : texto resaltado.
- `<small>` : texto secundario o de menor importancia.
- `<sup>` : superíndice.
- `<sub>` : subíndice.
- `<del>` : contenido eliminado.
- `<ins>` : contenido insertado.

También existen `<b>` e `<i>`, pero en general conviene priorizar etiquetas con más significado semántico, como `<strong>` y `<em>`.

Enlaces

Los enlaces se definen con la etiqueta `<a>`. El atributo `href` indica el destino.

```
<a href="https://www.google.com">Ir a Google</a>
```

Los enlaces son la base del hipertexto: permiten relacionar un recurso con otro.

Imágenes

Las imágenes se insertan con `<img>`. Dos atributos importantes son `src`, que indica el archivo, y `alt`, que describe la imagen.

```

```

El texto alternativo es importante para accesibilidad y para contextos donde la imagen no puede mostrarse.

Listas

HTML ofrece listas no ordenadas y ordenadas.

- `<ul>` : lista no ordenada.
- `<ol>` : lista ordenada.
- `<li>` : cada ítem de la lista.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Tablas

Las tablas se usan para representar información tabular, no para maquetar páginas.

- `<table>` : tabla completa.
- `<tr>` : fila.
- `<th>` : celda de encabezado.
- `<td>` : celda de datos.

```
<table>
  <tr>
    <th>Materia</th>
    <th>Nota</th>
  </tr>
  <tr>
    <td>Programación III</td>
    <td>8</td>
  </tr>
</table>
```

Formularios

Los formularios permiten ingresar y enviar datos.

- `<form>` : formulario.
- `<label>` : etiqueta descriptiva de un campo.
- `<input>` : campo de entrada.
- `<button>` : botón.

```
<form>
  <label for="nombre">Nombre</label>
  <input id="nombre" type="text">
  <button>Enviar</button>
</form>
```

Buenas prácticas iniciales

- Usar una estructura base completa con `<!DOCTYPE html>`, `<html>`, `<head>` y `<body>`.
- Escribir las etiquetas en minúscula.
- Indentar el código de forma consistente.
- Cerrar correctamente los elementos.
- Usar nombres de archivo simples y descriptivos.
- Preferir etiquetas semánticas cuando existan.
- Escribir texto alternativo útil en las imágenes.
- No usar HTML para resolver problemas de diseño que corresponden a CSS.

Idea clave: HTML no describe "cómo debería verse" algo, sino "qué es" ese algo dentro del documento. Aprender HTML no consiste solo en memorizar etiquetas, sino en representar información de manera ordenada y con significado.

5. Introducción a CSS

CSS significa **Cascading Style Sheets**. Es el lenguaje que describe la presentación visual de un documento HTML. Mientras HTML organiza el contenido y le da significado, CSS indica cómo debe verse ese contenido en pantalla.

Qué cosas controla CSS

Con CSS se puede decidir, entre otras cosas:

- Qué color tendrá un texto.

- Qué tipografía se utilizará.
- Cuánto espacio habrá entre elementos.
- Si un bloque tendrá borde, fondo o sombra.
- Cómo se distribuirán los elementos en la página.

Por qué conviene separar HTML y CSS

Separar contenido y presentación trae varias ventajas: mejora el mantenimiento, evita repetir estilos, permite reutilizar la misma hoja en varias páginas y hace posible cambiar el diseño sin tocar la estructura HTML.

Primer ejemplo

Un documento HTML puede mantener exactamente el mismo contenido, pero cambiar su apariencia si se le agrega una hoja de estilos.

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f4f4f4;  
}  
  
h1 {  
  color: #1d4ed8;  
}  
  
p {  
  color: #333333;  
}
```

La idea importante es que CSS no agrega contenido: modifica la forma en que el navegador lo presenta.

La regla CSS

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

- `p` es el selector.
- `color` y `font-size` son propiedades.
- `blue` y `18px` son valores.

Cada declaración termina en punto y coma y el conjunto de declaraciones se encierra entre llaves.

Comentarios en CSS

Los comentarios permiten dejar aclaraciones sin afectar el resultado visual. Se escriben entre `/*` y `*/`.

```
/* Estilo principal del encabezado */  
h1 {  
  color: navy;  
}
```

Formas de conectar CSS con HTML

Hay tres formas habituales de aplicar CSS:

- **En línea:** con el atributo `style` dentro de una etiqueta HTML.
- **Interno o integrado:** con una etiqueta `<style>` en el `<head>`.
- **Externo:** con un archivo `.css` enlazado desde HTML con `<link>`.

El enfoque externo es el más recomendado porque reutiliza estilos, reduce repetición, ordena el proyecto y además el navegador puede guardar la hoja en caché.

Ejemplos de conexión

```
<p style="color: red">Un texto</p>  
  
<style>  
  p {  
    color: blue;  
  }  
</style>  
  
<link rel="stylesheet" href="estilo.css">
```

Sintaxis básica

```
selector {  
  propiedad: valor;  
  propiedad: valor;  
}
```

La unidad fundamental de CSS es la regla. Todo el lenguaje se apoya en esa estructura básica.

Selectores CSS

Los selectores indican a qué elementos HTML se aplicará una regla.

- **Selector de elemento:** selecciona todas las etiquetas de un tipo, por ejemplo `p` o `body`.
- **Selector de clase:** selecciona elementos con una clase específica, por ejemplo `.tarjeta`.
- **Selector de id:** selecciona un elemento con un `id` concreto, por ejemplo `#encabezado`.
- **Selector universal:** `*`, aplica a todos los elementos.
- **Selector descendente:** por ejemplo `nav a`, para enlaces dentro de `nav`.
- **Agrupación de selectores:** por ejemplo `h1, h2, h3`, cuando varias etiquetas comparten estilo.

```
p {  
  color: #444;  
}  
  
.destacado {  
  background-color: yellow;  
}  
  
#encabezado {  
  color: navy;  
}  
  
nav a {  
  text-decoration: none;  
}  
  
h1, h2, h3 {  
  font-family: Georgia, serif;  
}
```

Un `id` debería ser único dentro del documento. Las clases, en cambio, pueden repetirse en muchos elementos.

Pseudoclases básicas

Las pseudoclases permiten aplicar estilos según el estado de un elemento.

```
a:hover {  
  color: orange;
```

```
}  
  
input:focus {  
  border-color: blue;  
}
```

- **:hover** se activa cuando el puntero pasa por encima.
- **:focus** se activa cuando el elemento recibe el foco.

Se usan mucho en enlaces, botones, formularios y menús.

Cascada, herencia y especificidad

La palabra “cascading” explica cómo decide el navegador qué reglas aplicar cuando varias afectan al mismo elemento.

- **Cascada:** varias reglas pueden competir y el navegador resuelve cuál usar.
- **Herencia:** algunas propiedades, como color o tipografía, pueden pasar del padre al hijo.
- **Especificidad:** un selector más específico suele imponerse sobre otro más general.

Esto explica por qué a veces un estilo “no se aplica”: puede estar siendo superado por otra regla de mayor especificidad.

Valores CSS

Las propiedades aceptan distintos tipos de valores. Dos grupos muy importantes al comenzar son colores y longitudes.

- **Colores:** por nombre, hexadecimal, `rgb()`, `rgba()` o `hsl()`.
- **Longitudes y porcentajes:** por ejemplo `px` y `%`.

```
h1 {  
  color: red;  
}  
  
p {  
  color: #333333;  
}  
  
.caja {  
  width: 80%;  
  margin: 20px;  
}
```

Propiedades CSS frecuentes

Al comenzar, conviene reconocer al menos estas propiedades:

- `color` : cambia el color del texto.
- `background-color` : cambia el color de fondo.
- `font-size` : tamaño de la letra.
- `font-family` : familia tipográfica.
- `text-align` : alineación del texto.
- `width` y `height` : dimensiones.
- `border` : borde del elemento.
- `margin` : espacio externo.
- `padding` : espacio interno.

```
body {  
  background-color: #fafafa;  
  color: #222;  
  font-family: Arial, sans-serif;  
}  
  
h1 {  
  text-align: center;  
}  
  
.caja {  
  width: 300px;  
  border: 1px solid #ccc;  
  margin: 20px;  
  padding: 16px;  
}
```

Buenas prácticas iniciales

- Preferir hojas de estilo externas.
- Usar nombres de clases claros y descriptivos.
- Evitar estilos en línea salvo casos excepcionales.
- Agrupar reglas relacionadas.
- No repetir estilos innecesariamente.
- Mantener una indentación consistente.

Ejemplo completo

El apunte cierra con un ejemplo simple de HTML y CSS trabajando juntos.

```
<!-- HTML -->
<h1 class="titulo">Programación III</h1>
<p class="descripcion">Introducción a HTML, CSS y JavaScript.</p>
<a class="enlace" href="#">Ver más</a>

/* CSS */
.titulo {
  color: #1d4ed8;
}

.descripcion {
  color: #333333;
}

.enlace {
  color: #0f766e;
}
```

Este tipo de ejemplo muestra de forma concreta cómo HTML aporta el contenido y CSS su apariencia.

Idea clave: CSS permite controlar la apariencia de un documento HTML. A medida que avances, a esto se le sumarán temas como modelo de caja, `display`, Flexbox y Grid.

6. Elementos HTML de bloque y en línea

Este tema sirve para entender cómo los elementos se distribuyen visualmente en la página antes de aplicar estilos más complejos. Algunos elementos ocupan su propia línea y otros se integran dentro de la misma línea de texto.

Elementos de bloque

Un elemento de bloque comienza en una línea nueva y, por defecto, ocupa todo el ancho disponible de su contenedor. Suelen usarse para organizar partes grandes de la estructura, como secciones, párrafos o listas.

Ejemplos: `div`, `p`, `h1` a `h6`, `section`, `article`, `ul`, `ol`, `li`, `form`, `table`.

Elementos en línea

Un elemento en línea no comienza en una línea nueva y ocupa solo el ancho necesario para mostrar su contenido. Se usa normalmente para marcar o modificar porciones pequeñas dentro de un bloque de texto.

Ejemplos: `span`, `a`, `strong`, `em`, `small`, `img`, `label`.

Aspecto	Bloque	En línea
Inicio en línea nueva	Sí	No
Ancho disponible	Ocupa todo por defecto	Solo lo necesario
Uso habitual	Estructura o agrupación grande	Marcado dentro del texto

`div` y `span`

Son dos contenedores genéricos muy usados. `div` es de bloque y `span` es en línea. Eso no significa que tengan semántica fuerte por sí mismos; más bien sirven para agrupar contenido cuando hace falta aplicar estilos o estructura auxiliar.

Relación con CSS

El material aclara algo importante: bloque y línea describen el comportamiento visual inicial o predeterminado, no una verdad absoluta del elemento. Con CSS, especialmente con la propiedad `display`, ese comportamiento puede cambiar. Esto muestra de nuevo la diferencia entre estructura y presentación.

Idea útil: cuando estudiás bloque y línea, no pienses solo en “cómo se ve”, sino en cómo el elemento participa en el flujo normal del documento.

Errores conceptuales frecuentes

- Creer que un elemento de bloque es “más importante”.
- Confundir comportamiento visual con significado semántico.
- Olvidar que CSS puede cambiar el comportamiento con `display`.

7. El modelo de caja

El modelo de caja es uno de los conceptos más importantes de CSS. Explica cómo el navegador calcula el espacio que ocupa un elemento. Si entendés el modelo de caja, entendés por qué algo mide lo que mide, por qué dos elementos quedan separados y por qué a veces una interfaz no entra donde esperabas.

La idea general

Muchos elementos HTML se representan como cajas rectangulares. Esa caja no es solo el contenido visible: también incluye zonas internas y externas que afectan el tamaño total.

Las cuatro áreas de la caja

- **Contenido:** zona donde aparece el texto, la imagen u otros elementos anidados.
- **Padding:** espacio interno entre el contenido y el borde.
- **Border:** borde que rodea el contenido y el padding.
- **Margin:** espacio externo respecto de otras cajas.

```
.tarjeta {  
  width: 300px;  
  padding: 16px;  
  border: 1px solid #ccc;  
  margin: 20px;  
}
```

Contenido

Es la parte central. Cuando se define `width` o `height` en el modelo estándar, esos valores se aplican primero al área de contenido. Esto es importante porque mucha gente cree que

`width: 300px` significa "todo el elemento mide 300 px", y no necesariamente es así.

Padding

Es el espacio interior entre contenido y borde. Sirve para que el contenido no quede pegado al borde. Visualmente suele hacer que un elemento "respire" mejor. Forma parte del tamaño visual del elemento.

Border

Es la línea que rodea el contenido y el padding. También ocupa espacio. Este detalle es muy importante porque a veces se lo olvida en los cálculos.

Margin

Es el espacio externo que separa una caja de otras. A diferencia del padding, no está dentro del borde. Su función principal es separar elementos entre sí.

box-sizing

Para simplificar el cálculo del tamaño real, CSS permite elegir cómo interpretar ancho y alto.

- `content-box` : es el valor por defecto. `width` y `height` se aplican solo al contenido, y luego se suman padding y borde.
- `border-box` : el ancho y el alto ya incluyen contenido, padding y borde.

Consecuencia práctica: con `border-box` es más fácil razonar tamaños, porque el valor declarado se parece más al tamaño final visible.

Propiedades abreviadas

En márgenes y paddings se pueden usar atajos. Con cuatro valores, el orden es: arriba, derecha, abajo, izquierda. Es un detalle técnico importante porque suele aparecer en ejercicios y preguntas de interpretación de código.

Relación con bloque y línea

Todos estos conceptos se entienden mejor recordando el tipo de elemento. Los elementos de bloque aceptan naturalmente ancho, alto, márgenes verticales, padding y bordes. Los elementos en línea también participan del modelo de caja, pero no siempre responden igual a ancho y alto. Por eso no se puede generalizar que "todos los elementos CSS se comportan igual".

Colapso de márgenes

Los márgenes verticales de algunos elementos de bloque pueden colapsar. Esto significa que dos márgenes adyacentes no siempre se suman. En muchos casos, el navegador usa el valor mayor. Es un comportamiento que puede sorprender, así que conviene al menos saber que existe, aunque no se profundice en todos sus casos.

Errores frecuentes

- Pensar que `width` siempre es el ancho total visible.
- Usar `margin` cuando el problema se resuelve con `padding`.
- Olvidar que el borde también ocupa espacio.
- No considerar `box-sizing` al calcular tamaños.

8. Introducción a JavaScript

JavaScript es el lenguaje de programación que agrega lógica y comportamiento a las aplicaciones web. Si HTML define la estructura y CSS la apariencia, JavaScript permite tomar decisiones, repetir acciones, guardar datos, transformarlos y organizar código reutilizable.

Por qué es central en desarrollo web

Porque permite programar la parte dinámica de la aplicación. Con JavaScript se pueden validar datos, transformar información, calcular resultados, organizar funciones reutilizables y conectar distintas partes del sistema. El material aclara que en esta etapa el foco está en fundamentos del lenguaje, no en DOM ni eventos.

Cómo incluir JavaScript en HTML

- Escribiéndolo dentro de una etiqueta `<script>` en el mismo documento.
- Cargando un archivo `.js` externo, que es la opción recomendada para proyectos más ordenados.

La recomendación de usar archivos separados se conecta con la misma idea que vimos en CSS: separar responsabilidades mejora mantenimiento y claridad.

Variables

Una variable es un nombre que referencia un valor. Permite guardar información para usarla después. En JavaScript, el tipo del valor se determina en tiempo de ejecución, por lo que el lenguaje es dinámicamente tipado.

Tipos de datos básicos

- **string:** texto.
- **number:** números enteros o decimales.
- **boolean:** `true` o `false`.
- **null:** ausencia intencional de valor.
- **undefined:** valor no asignado o no definido.

`var`, `let` y `const`

Las tres palabras sirven para declarar variables, pero no se comportan igual. El material insiste en compararlas porque es un punto clásico de evaluación.

Característica	var	let	const
Alcance	Función	Bloque	Bloque
Redeclaración en mismo bloque	Sí	No	No
Reasignación	Sí	Sí	No
Inicialización obligatoria	No	No	Sí

En código moderno se recomienda usar `const` por defecto y `let` cuando el valor deba cambiar. `var` queda asociado a código más antiguo o a casos muy específicos.

Condicionales

Los condicionales permiten ejecutar diferentes bloques de código según una condición. La estructura básica es `if`, y puede complementarse con `else` y `else if`.

```
if (nota >= 6) {  
  console.log("Aprobado");  
} else {  
  console.log("Desaprobado");  
}
```

La idea conceptual es sencilla: el programa evalúa una expresión booleana y decide qué camino seguir.

Bucles

Los bucles permiten repetir instrucciones. Son útiles cuando hay tareas repetitivas o cuando se necesita recorrer colecciones de datos.

- `for`: se usa mucho cuando se sabe cuántas repeticiones habrá o cuando se recorre por índice.
- `while`: se usa cuando la repetición depende de una condición que se evalúa antes de cada vuelta.

Funciones

Una función es un bloque reutilizable de código. Permite agrupar instrucciones con un propósito definido. Puede recibir parámetros y devolver un resultado con `return`.

```
function saludarPersona(nombre) {  
  return "Hola, " + nombre;  
}
```

La ventaja de las funciones es que evitan repetir lógica, hacen el código más legible y permiten dividir problemas grandes en partes pequeñas.

Funciones flecha

Las funciones flecha son una forma más corta de escribir funciones. Se usan mucho en JavaScript moderno, especialmente como callbacks.

```
const cuadrado = numero => {  
  return numero * numero;  
};
```

El material también compara función tradicional y función flecha para remarcar diferencias de sintaxis y uso habitual.

Arreglos

Un arreglo permite guardar varios valores en una misma estructura. Cada valor ocupa una posición, llamada índice, que comienza en 0. Este tema se conecta directamente con el siguiente apunte, donde se estudian los métodos de arreglos.

Objetos

Un objeto agrupa datos mediante propiedades. A diferencia del arreglo, donde el acceso se hace por posición, en el objeto el acceso se hace por nombre de propiedad.

```
let alumno = {  
  nombre: "Valentina",  
  edad: 22,  
  activo: true  
};
```

Módulos

Cuando un programa crece, conviene dividirlo en varios archivos. En JavaScript eso se resuelve mediante módulos. La idea general es separar responsabilidades, reutilizar código y evitar mezclar todo en un solo archivo enorme.

Resumen conceptual: JavaScript toma los datos, decide qué hacer con ellos, repite tareas cuando hace falta y organiza esa lógica en funciones, objetos, arreglos y módulos.

9. Métodos de arreglos

Los arreglos ocupan un lugar central en JavaScript porque permiten trabajar con colecciones de datos. El apunte muestra que muchas operaciones que podrían hacerse con `for` o `while` pueden expresarse de manera más clara usando métodos propios del arreglo.

Recordatorio: qué es un arreglo

Es una estructura que agrupa varios valores en una sola variable. Cada elemento tiene un índice, comenzando en 0. Esta idea es básica, pero hace falta tenerla muy firme para entender el resto.

Método y callback

Muchos métodos de arreglos reciben una función como argumento. A esa función se la llama **callback**. El callback se ejecuta una vez por cada elemento del arreglo, o según la lógica del método.

```
const numeros = [1, 2, 3];

numeros.forEach(numero => {
  console.log(numero);
});
```

Acá, `forEach` es el método y la función flecha es el callback.

No todos devuelven lo mismo

Este es uno de los puntos más importantes del tema. Algunos métodos devuelven un nuevo arreglo, otros devuelven un booleano, otros un valor único y otros modifican el arreglo original. Por eso no alcanza con "saber usarlos": también hay que entender qué produce cada uno.

Método	Qué hace	Qué devuelve	Idea clave
<code>forEach</code>	Recorre el arreglo.	<code>undefined</code>	Sirve para ejecutar acciones, no para transformar y guardar

Método	Qué hace	Qué devuelve	Idea clave
			resultado.
<code>map</code>	Transforma cada elemento.	Nuevo arreglo	Mantiene la misma cantidad de elementos.
<code>filter</code>	Filtra por condición.	Nuevo arreglo	Puede devolver menos elementos que el original.
<code>some</code>	Verifica si alguno cumple.	Booleano	Responde "existe al menos uno".
<code>every</code>	Verifica si todos cumplen.	Booleano	Responde "todos cumplen".
<code>find</code>	Busca el primer elemento que cumple.	Elemento o <code>undefined</code>	No devuelve todos, solo el primero.
<code>findIndex</code>	Busca la posición del primer elemento que cumple.	Índice o <code>-1</code>	Es útil cuando importa la posición.
<code>reduce</code>	Acumula en un único resultado.	Valor final	Sirve para sumar, contar, promediar o construir estructuras.
<code>includes</code>	Verifica existencia de un valor.	Booleano	Muy legible para "está / no está".
<code>indexOf</code>	Busca posición de un valor.	Índice o <code>-1</code>	Sirve si necesitás la ubicación, no solo saber si existe.
<code>slice</code>	Copia una parte del arreglo.	Nuevo arreglo	No modifica el original.
<code>concat</code>	Une arreglos.	Nuevo arreglo	Tampoco modifica los originales.
<code>join</code>	Une elementos en un texto.	String	Convierte una lista en una cadena.

Explicación de los más importantes

forEach

Recorre el arreglo y ejecuta una función por cada elemento. Es ideal para mostrar datos o producir efectos secundarios, como imprimir en consola. No devuelve un arreglo nuevo útil para seguir transformando.

map

Genera un nuevo arreglo a partir de transformar cada elemento. Si tenías una lista de precios y querés otra lista con IVA incluido, `map` es una buena elección. La idea clave es que conserva la cantidad de elementos.

filter

Construye un nuevo arreglo con los elementos que cumplen una condición. El callback debe devolver algo interpretable como verdadero o falso. Si devuelve verdadero, el elemento queda; si no, se descarta.

some y every

Ambos responden preguntas lógicas sobre el conjunto. `some` pregunta "¿hay al menos uno que cumpla?". `every` pregunta "¿todos cumplen?". Conviene compararlos juntos porque se entienden por oposición.

find y findIndex

`find` devuelve el primer elemento que cumple una condición. `findIndex` devuelve su posición. Si no encuentran nada, devuelven `undefined` y `-1` respectivamente.

reduce

Es uno de los métodos más potentes porque reduce todo el arreglo a un único resultado. Ese resultado puede ser un número, un string, un objeto, otro arreglo o casi cualquier estructura. Suele usarse para sumas, promedios, acumuladores y agrupamientos.

Métodos que modifican el arreglo original

El apunte también remarca que no todos los métodos preservan el arreglo intacto.

- `push` : agrega uno o más elementos al final.
- `pop` : quita el último elemento.
- `shift` : quita el primer elemento.
- `unshift` : agrega uno o más elementos al principio.

Esta distinción es importante porque en algunos ejercicios el arreglo original debe conservarse, y en otros no importa si se modifica.

Encadenamiento de métodos

Una gran ventaja de varios métodos es que se pueden encadenar. Eso permite describir procesos de transformación en pasos claros.

```
const resultado = numeros
  .filter(numero => numero % 2 === 0)
  .map(numero => numero * 10);
```

Primero se filtran los pares. Después, sobre ese resultado, se transforman multiplicándolos por 10. El orden importa porque cada método trabaja sobre lo que devuelve el anterior.

Regla práctica: si querés transformar, pensá en `map`. Si querés quedarte con algunos, pensá en `filter`. Si querés validar una condición, pensá en `some` o `every`. Si querés resumir todo en un único valor, pensá en `reduce`.

10. Conexiones entre temas

Una parte importante de estudiar bien no es memorizar cada apunte por separado, sino ver cómo se conectan.

- La **Web** es el contexto general donde viven páginas y aplicaciones.
- **HTML** estructura el contenido que el navegador recibe.
- **CSS** controla la presentación visual de esa estructura.
- Los conceptos de **bloque, línea y modelo de caja** explican cómo se dibujan y distribuyen esos elementos.
- **JavaScript** agrega lógica y comportamiento a la interfaz.
- Los **métodos de arreglos** son herramientas concretas para trabajar con colecciones de datos dentro de JavaScript.
- **Git y ramas** no forman parte de la página en sí, pero son indispensables para organizar el desarrollo del proyecto.

Mirada global: Git organiza el trabajo del programador; la Web define el entorno de ejecución; HTML arma la estructura; CSS define la forma visible; JavaScript define la lógica; y los métodos de arreglos ayudan a procesar datos dentro de esa lógica.

11. Repaso rápido para el parcial

Preguntas que conviene poder responder con tus palabras

- ¿Qué problema resuelve un sistema de control de versiones?
- ¿Qué diferencia hay entre VCS local, centralizado y distribuido?
- ¿Qué significa que Git trabaje con instantáneas?
- ¿Cuáles son los estados principales de un archivo en Git?
- ¿Qué es una rama y qué representa HEAD?
- ¿Qué significa cliente y servidor en la Web?
- ¿Qué diferencia hay entre Internet y Web?
- ¿Qué función cumple HTML y por qué no es un lenguaje de programación?
- ¿Qué diferencia hay entre etiqueta, elemento y atributo?

- ¿Qué es CSS y qué significa cascada?
- ¿Cuál es la diferencia entre elementos de bloque y elementos en línea?
- ¿Cuáles son las cuatro áreas del modelo de caja?
- ¿Cuándo usar `let` y cuándo `const` ?
- ¿Qué hace `map` , qué hace `filter` y qué hace `reduce` ?

Frase final de repaso: un proyecto web se desarrolla mejor cuando Git ordena su historia, la Web provee el entorno, HTML estructura, CSS presenta y JavaScript procesa datos y comportamiento con herramientas como funciones, arreglos y métodos de colección.